

Integrating AI into Operating Systems: an Overview

Sun Shuo, Chenyu Tong, Houpu Yu

November 6, 2025

Abstract

Large language models have recently demonstrated great capability of solving complex natural language processing tasks, and the numerous other machine learning techniques showed potential of enhancing basic operating system functionalities. The great power of LLMs and other machine learning methods have led to a series of innovations in operating system designs. This overview provides a brief examination of AI OS concepts and developments, covering the conceptual frameworks, concrete architectures, and the various contexts from cloud to edge where an AI-enhanced OS could be applied. We will discuss how LLMs augment traditional OS functions, the technical challenges in building such systems, and future directions toward modular, hardware-fused intelligent operating systems, as well as shortcomings and problems in current AI OS designs.

1 Introduction

The integration of artificial intelligence (AI) into operating systems (OSes) is reshaping both the design goals and the implementation strategies of systems software. On the one hand, AI techniques—ranging from lightweight prediction models to large language models (LLMs)—promise to improve classical OS concerns such as scheduling, memory management, and storage I/O. On the other hand, emerging AI workloads demand new OS abstractions and control planes that better accommodate heterogeneous accelerators, dynamic workloads, and data-driven decision loops across cloud and edge. Recent surveys highlight this bidirectional relationship: AI can enhance OS performance, security, and efficiency, while OS innovations enable more effective deployment of AI applications [1].

A notable trend in current literature is the growth of work coupling LLMs with OSes. A systematic review classifies research across *AI for OS* and *OS for AI*, and explicitly tracks a rapid rise of publications on “LLM as OS” and related topics since 2023 [1]. In parallel, position papers and systems research advocate rethinking OS structure towards disaggregation and service orientation, with machine learning (ML) deeply embedded in resource management [2].

This overview synthesizes these developments and organizes them into (i) conceptual frameworks that inspire AI-augmented OS design, (ii) implemented architectures that prototype concrete mechanisms, (iii) synergies between AI and core OS functions, and (iv) open challenges and future directions. We draw exclusively on the provided references, which collectively span surveys [1], architectural visions [2], and systems that bring ML and LLMs into the OS stack [3, 4, 5, 6, 7, 8].

2 Background

Classical OSes were engineered around monolithic or microkernel designs and around a single-machine abstraction. As software increasingly runs over distributed and heterogeneous resources—cloud data centers, edge servers, personal devices, and IoT nodes—the control plane must adapt to frequent changes in hardware, APIs, and workload patterns [2]. ServiceOS argues that monolithic structures hinder such adaptation: resource management remains tightly coupled and inflexible, particularly for distributed resources [2].

In response, the systems community has embraced decoupling and service orientation. The ServiceOS vision crystallizes three principles: *resource disaggregation*, *service-oriented resource provisioning*, and *learning-based allocation and scheduling* [2]. Disaggregation allows compute, memory, storage, and accelerators to evolve independently and be orchestrated over high-bandwidth, low-latency fabrics; memory can be decoupled along local/remote and physical/virtual dimensions, with RDMA-based control/data channels supporting remote access [2]. Service-oriented provisioning leverages containers, microservices, and FaaS to rapidly instantiate functions and reconfigure resources, while navigating security and isolation trade-offs relative to VMs [2]. At the same time, the vision acknowledges practical challenges—from GPU virtualization intricacies to container networking overheads—that must be engineered around [2].

AI-driven operating systems also intersect with human factors and personalization. SmartOS shows that user preferences vary over time and that OS policies like the Linux Completely Fair Scheduler may not align with what the user currently values. A reinforcement learning (RL) layer can learn user intent and adjust resource allocations accordingly [8]. Survey evidence similarly emphasizes how AI techniques are being applied to OS tuning and security, and points to LibOS paradigms and LLM-driven interfaces as promising building blocks for intelligent OSes [1].

3 Conceptual Frameworks

ServiceOS and the service-orientation of resources. ServiceOS proposes to expose all resources as services and to rely on learning-based mechanisms to schedule and provision them dynamically [2]. The argument is that modular, composable services ease evolution and enable ML to tune allocations as workload and environment shift. This concept works in tandem with disaggregated hardware and high-speed interconnects, and it anticipates container/VM hybrids and specialized runtimes to balance isolation and agility [2].

LLM-as-OS-module. Rather than replace kernels, recent work embeds LLMs into the control plane to interpret unstructured device descriptions and to synthesize policy decisions. LLaMaS (“Herding LLaMaS”) exemplifies this direction: it leverages an LLM frontend to parse textual device specifications, producing embeddings that help a backend recommend OS tactics (e.g., memory tiering decisions) without hand-crafting device-specific code [3]. The systematic review classifies such systems under *LLM AS OS* and stresses their potential to streamline adaptation as hardware evolves [1].

Agentic, cloud-hosted OS mediators. PerOS envisions a personalized, self-adapting OS that uses an LLM-based agent as the user’s proxy, spanning OS, filesystem, Git, web, and cloud tools. The agent translates high-level natural language tasks into sequences of tool invocations, maintains

task-level memory, and iteratively refines plans based on feedback [4]. A declarative interface exposes capabilities (e.g., file operations, browsing, coding) and the agent composes them in response to user goals [4].

OS abstractions around LLM memory. MemGPT proposes OS-like memory management for LLMs themselves: a *virtual context manager* swaps relevant state between the narrow working context and a larger external memory, enabling long-lived, multi-session agency while respecting token limits [5]. This reframes the OS/AI boundary: not only should OSes manage resources for AI, but OS-like ideas (paging, hierarchies, buffers) can manage AI agents.

Human-centered adaptive OS. SmartOS positions the OS as a learner of user intent, with an RL feedback loop that prioritizes foreground tasks and dynamically tunes CPU, memory, I/O, and network bandwidth [8]. This stands in contrast to static, one-size-fits-all policies and illustrates an end-to-end adaptation path within a commodity OS.

Survey synthesis. The systematic review argues for coupling LLM-driven interfaces with modular OS architectures such as LibOS, highlighting opportunities to personalize, to harden security via AI-based analysis, and to reduce developer effort across the OS lifecycle [1].

4 Implemented Architectures

Predictive I/O at microsecond scale. LinnOS uses a compact neural network inside the I/O path to predict per-request latency on black-box SSDs, allowing the OS to detect problematic requests early and steer around slow paths. A compressed model runs in 4–6 microseconds per I/O, minimizing decision overhead while improving predictability and throughput [6]. Crucially, the design includes misprediction mitigation to avoid harming fast I/O [6].

Latency-predictable multi-DNN scheduling. NeuOS targets autonomous systems that run multiple DNNs concurrently under real-time constraints. It introduces a multi-dimensional optimization across model accuracy, DVFS settings, and batch sizes to meet tail-latency SLOs, and implements an interference-aware scheduler to coordinate GPU/CPU resources [7]. The result is a systems framework that trades controlled accuracy for predictable latency.

User-adaptive resource control in a commodity OS. SmartOS implements an RL-based controller in Linux that learns what the user values in the moment and shifts allocations across CPU, memory, I/O, and network to meet perceived priorities. The prototype demonstrates that such learning converges rapidly and can improve interactive experiences relative to fixed heuristics [8].

LLM module for OS tactics. Herding LLaMaS demonstrates an LLM-assisted OS module that parses textual device specs and maps them to embeddings used by a backend to recommend OS decisions such as memory-tier placements [3]. The goal is to reduce engineering overhead when integrating new, rapidly evolving hardware.

Agentic, end-to-end OS proxy. PerOS prototypes an LLM agent that chains OS, code, and cloud tools behind a single declarative interface, persists context across sessions, and iteratively executes high-level tasks for the user [4].

LLM memory as an OS problem. MemGPT presents a running system with paging-like mechanisms for LLM context management, enabling longer-horizon assistive agents [5].

5 Synergies: AI and core system functions

Scheduling and policy learning. RL- or ML-driven scheduling enables the OS to deviate from static heuristics and tailor allocations to user intent or workload phases. SmartOS demonstrates such adaptation in Linux [8], while the survey catalogues broader uses of AI for OS tuning (scheduling, energy, memory) and identifies a growing body of evidence that learning can outperform global, static policies [1].

Storage and I/O predictability. LinnOS shows that small, carefully engineered models can meet microsecond budgets and yield meaningful control decisions (e.g., bypassing slow SSD paths), translating learned predictions into OS-level actions [6].

Memory and accelerator management. Disaggregated memory and accelerator pools complicate placement and migration; the ServiceOS vision addresses this with service exposure and ML-driven orchestration [2]. Concurrently, LLM-oriented memory management (MemGPT) applies OS ideas back to AI agents, indicating a two-way transfer of concepts [5].

Security and personalization. The survey argues that LLMs can help enforce policies and detect threats while also personalizing interfaces; however, integrating these capabilities at the OS level raises correctness and safety questions [1].

Interfaces and orchestration. PerOS and LLaMaS illustrate how LLMs can mediate between high-level intent and low-level system actions. PerOS focuses on multi-tool orchestration and persistent task memory [4], whereas LLaMaS uses LLMs as policy advisors based on textual device knowledge [3]. Together they suggest an “intent-to-execution” stack within or adjacent to the OS.

6 Challenges

Structural inertia and kernel integration. ServiceOS critiques monolithic kernels for their difficulty in evolving to distributed, heterogeneous environments [2]. Refactoring an existing kernel around services, safe isolation, and ubiquitous virtualization is a long-term endeavor, with practical hurdles in GPU virtualization, container networking, and device service exposure [2].

Reliability of learned decisions. Learning-generated adaptations that touch low-level functions (interrupts, memory mappings, filesystems) must be highly reliable; inaccurate tuning risks fatal errors [2]. Similarly, LLMs may misinterpret specs or hallucinate, so systems like LLaMaS and PerOS should contain guardrails, validation layers, and rollback mechanisms.

Overheads and tight time budgets. AI components must operate within strict latency/throughput envelopes. LinnOS shows the kind of microsecond-scale budgets a model must respect in the I/O path [6], while NeuOS demonstrates that multi-DNN workloads require interference-aware scheduling and multi-dimensional optimization to meet SLOs [7].

Privacy, security, and provenance. Agentic OS proxies raise questions about data handling, capability scoping, and provenance of actions. The survey underscores the necessity of secure integration and lightweight algorithms suitable for constrained environments [1].

Developer experience and evolvability. Even with LLM assistance, mapping ambiguous intent to safe, correct system actions is difficult. ServiceOS acknowledges that many techniques remain immature and that building an executable system at scale will require substantial engineering [2].

7 Shortcomes and future directions

Previous works have shown that LLMs and other machine learning techniques can boost OS performance and usability, but progress toward a cohesive *AI-integrated OS* remains incremental. Based on the evidence and prototypes surveyed earlier [1, 2, 6, 7, 8, 3, 4, 5], we summarize the major shortcomes and chart directions that could turn point solutions into a robust platform.

Lack of a data-fluent substrate A consistent pain point is data accessibility. Contemporary AI services often depend on user data that is scattered across siloed applications, each with its own stores, permissions, and APIs. The result is redundant copies, brittle integrations, and a reliance on ad-hoc commercial partnerships to bridge gaps—friction that hinders both developers and users. A future AI-ready OS should expose a *first-class, policy-aware data plane*: system-managed personal and organizational data spaces with uniform access semantics (discover, query, subscribe, mutate), capability-scoped authorization, and auditable provenance. Service-oriented ideas [2] already encourage factoring resources into independently evolvable services; applying the same discipline to *data*—not just compute and memory—would let agents and system services consume live, consistent views without proliferating private caches. Agentic systems such as PerOS [4] and memory-centric designs like MemGPT [5] highlight the need for durable, searchable state beneath LLMs; making that substrate an OS primitive rather than an app-level afterthought is a natural next step.

Ununified information formats and semantics Even when data is accessible, it is often semantically incompatible. Logs, calendars, documents, contacts, tasks, and device telemetry each use divergent schemas that frustrate global reasoning about user behavior, context, and intent. Without a shared semantic layer, cross-application analytics degrade into brittle ETL pipelines. We argue for an OS-level *typed event and entity schema* with versioning and per-field policies. Concretely: define core, extensible types (e.g., `CalendarEvent`, `Message`, `Task`, `File`, `DeviceSpec`) and standardize lifecycle signals (created/updated/accessed), privacy labels, and merge/conflict rules. Apps register adapters to map their internal models to these types. Such unification would directly benefit LLM-in-the-loop modules: LLaMaS [3] shows promise in interpreting textual device

specs, but richer, typed metadata would further ground LLM recommendations; SmartOS-style preference learning [8] could incorporate standardized signals to learn across application boundaries.

Fragmented user experience and the absence of intent-centric interfaces Today, each AI application ships its own UI, data model, and command surface. When functionally identical capabilities coexist (e.g., multiple calendar apps), events and actions fragment across silos, leading to duplicated entries, missed reminders, and inconsistent automation. A future OS should provide a *monolithic experience for intents, not for apps*: a unified, system-owned interface where users express goals (“schedule a review with the team next week”), and where multiple providers can fulfill the same capability behind a consistent contract. PerOS’s tool-composition approach [4] points the way: the OS can expose a declarative *intent registry* (schedule, message, summarize, translate, file-search, code-run) and adjudicate among providers based on policy, context, and user preference. This unifies UX without stifling ecosystem diversity, and it gives LLM agents a stable, auditable surface to act on.

Bridging micro- and macro-models safely Many promising results live at opposite ends of the spectrum: tiny, specialized models in tight loops (e.g., LinnOS’s microsecond-scale I/O predictors [6]) and large, agentic models orchestrating tools (e.g., PerOS [4]). The gap between these layers is under-specified: how do high-level intents compile into low-level, real-time decisions with safety guarantees? A practical direction is a *tiered control architecture*: (i) kernel/driver micro-models with hard budgets and formally specified fail-safes; (ii) user-space controllers that compose tactics and verify guard conditions; (iii) LLM agents that plan and explain at human timescales, confined by capabilities. NeuOS’s latency-aware optimization [7] and SmartOS’s human-in-the-loop adaptation [8] offer building blocks for such layering.

Reliability, provenance, and policy compliance for LLM-in-the-loop LLMs can misinterpret specs or hallucinate actions. Systems like LLaMaS [3] and PerOS [4] therefore need verifiable grounding. We see three converging requirements: (1) *spec-grounded reasoning*, where model outputs are checked against authoritative artifacts (typed schemas, device capabilities, OS policies); (2) *capability-scoped execution*, where every action flows through least-privilege tokens and yields machine-auditable receipts; and (3) *counterfactual validation*, where candidate plans are simulated against policy engines before execution. The systematic review [1] emphasizes security and lightweight algorithms for constrained settings; integrating these checks into the OS substrate rather than each application will lower risk and developer burden.

Developer experience and evolvability Even with better data and interfaces, developers face moving targets: evolving accelerators, changing APIs, and shifting user expectations. ServiceOS’s service orientation [2] helps, but developers also need *OS-hosted scaffolding*: schema registries; policy testbeds; synthetic, privacy-preserving datasets; and standard evaluation harnesses for AI-augmented subsystems (storage, scheduling, networking). Publishing *reproducible benchmarks*—latency/throughput for micro-models [6], SLO conformance under interference [7], preference-tracking stability [8], agent task success rates [4, 5]—will turn one-off demos into comparable progress.

A concrete roadmap *Near term*: (i) introduce a system data plane with capability-scoped APIs and typed schemas; (ii) standardize an intent registry and provider contracts; (iii) pilot

micro-models in latency-critical paths (I/O, scheduling) with verifiable fallbacks [6, 7]. *Mid term:* (iv) integrate LLM agents as first-class orchestrators confined by the OS (PerOS-style) [4]; (v) adopt spec-grounded validation for LLM suggestions (LLaMaS-style) [3]; (vi) extend SmartOS-like preference learning using unified signals [8]. *Long term:* (vii) co-design with disaggregated hardware and remote memory/services [2]; (viii) elevate memory abstractions for agents (MemGPT) to OS primitives for persistent, queryable context [5].

Illustrative scenario: calendars without chaos Consider a device with three calendar apps. In today’s model, events fragment across silos. In the proposed architecture, all calendars read/write the system’s `CalendarEvent` type in the shared data plane; the OS deduplicates and resolves conflicts using declared merge rules; an LLM agent invokes the `schedule` intent through the registry and the OS chooses a provider per policy; SmartOS-style signals feed back user preferences (e.g., preferred venue, typical durations) to improve scheduling [8, 4]. The user sees one coherent schedule; providers compete on value, not on data custody.

Closing the gap In short, the path forward is not merely “more AI inside the OS,” but “OS primitives that make AI—and apps—data-fluent, semantically unified, intent-centric, and safe.” The surveyed systems provide persuasive pieces of this puzzle [1, 2, 6, 7, 8, 3, 4, 5]. Turning them into a dependable, widely deployable stack will require disciplined semantics for data, capabilities, and intents; rigorous evaluation; and layered designs that let micro-models and macro-models cooperate without compromising correctness.

References

- [1] Y. Zhang, X. Zhao, J. Yin, L. Zhang, and Z. Chen. Operating System and Artificial Intelligence: A Systematic Review. *arXiv:2407.14567*, 2024.
- [2] X. Liu, S. Wang, Y. Ma, Y. Zhang, Q. Mei, Y. Liu, and G. Huang. Operating Systems for Resource-adaptive Intelligent Software: Challenges and Opportunities. *ACM Transactions on Internet Technology*, 21(2):Article 27, 2021.
- [3] A. K. Kamath and S. Yadalam. Herding LLaMaS: Using LLMs as an OS Module. *arXiv:2401.08908*, 2024.
- [4] H. Hè. PerOS: Personalized Self-Adapting Operating Systems in the Cloud. *arXiv:2404.00057*, 2024.
- [5] C. Packer, V. Fang, S. G. Patil, *et al.* MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.
- [6] M. Hao, L. Toksöz, N. Li, *et al.* LinnOS: Predictability on Unpredictable Flash Storage with a Light Neural Network. In *OSDI’20*, 2020, pp. 173–190.
- [7] S. Bateni and C. Liu. NeuOS: A Latency-Predictable Multi-Dimensional Optimization Framework for DNN-driven Autonomous Systems. In *USENIX ATC’20*, 2020, pp. 371–385.
- [8] S. Goodarzy, M. Nazari, R. Han, E. Keller, and E. Rozner. SmartOS: Towards Automated Learning and User-Adaptive Resource Allocation in Operating Systems. In *APSys’21*, 2021.